PYTHON ACTIVITY-BASED LEARNING

Another Set: 10 Different Case Studies

Student Name	Amrutha Yanaganti
Roll Number	26000919
Branch	CSM
Section	2

Selected Case Studies

1. Student Performance Trend Analysis
2. Personal Shopping Budget Planner
3. Repeated Word and Phrase Detection
4. Tuple-Based Employee Shift Conflict Detection
5. Food Waste Monitoring System
6. Library Borrowing and Fine Reconciliation
7. Manufacturing Quality Control Matrix
8. Multi-Branch Sales Matrix Comparison
9. Weekly Electricity Consumption Analysis
10. Campus Transport Capacity Planner

This is a separate set of examples and avoids repeating the ten topics from the previous PDF.
Rules and values are illustrative academic assumptions.

1. Student Performance Trend Analysis

Problem Statement

• Analyse a student's marks across multiple tests, calculate averages and percentage changes, and identify whether performance is improving or declining.

Objectives

• Calculate the average mark.

• Compare the first and latest test marks.

• Classify the performance trend.

Assumptions

• Marks are percentages from 0 to 100.

• At least two test results are required.

• An increase of more than 2 marks is improving, a decrease of more than 2 is declining, otherwise performance is stable.

Algorithm

1. Read the student's test marks.

2. Validate the number and range of marks.

3. Calculate the average, first mark, and latest mark.

4. Compare latest and first marks using the stated threshold.

5. Display the summary and trend.

Flowchart

START → Read test marks → Validate marks → Calculate average and change → Classify trend → Display summary → STOP

Pseudocode

```
BEGIN
  READ marks
  IF fewer than 2 marks OR any mark outside 0..100:
    DISPLAY invalid input
  ELSE
    average = sum(marks) / length(marks)
    change = last mark - first mark
    IF change > 2: trend = IMPROVING
    ELSE IF change < -2: trend = DECLINING
    ELSE: trend = STABLE
    DISPLAY average, change, trend
END
```

Python Source Code

```
marks = [62, 68, 71, 78]

if len(marks) < 2 or any(m < 0 or m > 100 for m in marks):
    print("Invalid marks")
else:
    average = sum(marks) / len(marks)
    change = marks[-1] - marks[0]

    if change > 2:
        trend = "IMPROVING"
    elif change < -2:
        trend = "DECLINING"
    else:
        trend = "STABLE"

    print("Test marks:", marks)
    print(f"Average: {average:.2f}")
    print("Change from first to latest:", change)
    print("Performance trend:", trend)
```

Test Cases

Test Input	Expected Result	Purpose
[62, 68, 71, 78]	Average 69.75; change +16; IMPROVING	Improving trend
[80, 75, 70]	Average 75; change -10; DECLINING	Declining trend

Test Input	Expected Result	Purpose
[70, 71, 71]	Change +1; STABLE	Stable trend

Conclusion

The program uses lists and simple calculations to summarize test performance and identify a clear trend.

2. Personal Shopping Budget Planner

Problem Statement

• Calculate the cost of a shopping list, compare it with a fixed budget, and report the remaining balance or amount exceeding the budget.

Objectives

• Calculate the total cost of selected items.

• Compare the total with the available budget.

• Display savings or the amount over budget.

Assumptions

• Prices and quantities cannot be negative.

• The sample budget is ■3,000.

• The total is the sum of each item's price multiplied by quantity.

Algorithm

1. Set the budget and item records.

2. Validate prices and quantities.

3. Calculate each item's cost and the total.

4. Subtract total from budget.

5. Display whether the shopping plan is within budget.

Flowchart

START → Set budget and items → Validate values → Calculate item totals → Compare total with budget → Display balance/status → STOP

Pseudocode

```
BEGIN
  total = 0
  FOR each item
    IF price < 0 OR quantity < 0: DISPLAY invalid
    ELSE total = total + price * quantity
  balance = budget - total
  IF balance >= 0: DISPLAY within budget and balance
  ELSE DISPLAY over-budget amount
END
```

Python Source Code

```
budget = 3000
items = [
    ("Shoes", 1200, 1),
    ("Notebook", 80, 4),
    ("Headphones", 900, 1),
    ("Water bottle", 250, 2)
]
total = 0
valid = True

for name, price, quantity in items:
    if price < 0 or quantity < 0:
        print("Invalid item:", name)
        valid = False
    else:
        cost = price * quantity
        total += cost
        print(f"{name}: Rs.{cost}")

if valid:
    balance = budget - total
    print("Total shopping cost: Rs.", total)
    if balance >= 0:
        print("Within budget. Remaining: Rs.", balance)
    else:
        print("Over budget by: Rs.", -balance)
```

Test Cases

Test Input	Expected Result	Purpose
Budget ■3,000; total ■2,960	Within budget; ■40 remaining	Within-budget calculation
Budget ■3,000; total ■3,500	Over budget by ■500	Over-budget calculation
Negative quantity	Invalid item	Validation

Conclusion

The planner helps estimate shopping costs and compare expenses against a budget using loops and arithmetic.

3. Repeated Word and Phrase Detection

Problem Statement

• Analyse a sentence or paragraph, count word occurrences without case sensitivity, and display words that appear more than once.

Objectives

• Normalize text to lowercase.

• Count word frequencies.

• Identify repeated words.

Assumptions

• Words are separated by spaces or punctuation.

• For this simple program, common punctuation is removed before counting.

• The program reports each repeated word once with its frequency.

Algorithm

1. Read the text and convert it to lowercase.

2. Replace common punctuation with spaces.

3. Split the text into words.

4. Count each word using a dictionary.

5. Display words whose counts exceed one.

Flowchart

START → Read text → Normalize case and punctuation → Split into words → Count occurrences → Display repeated words → STOP

Pseudocode

```
BEGIN
  READ text
  text = lowercase(text)
  replace punctuation with spaces
  words = split(text)
  counts = empty dictionary
  FOR each word in words:
    counts[word] = counts.get(word, 0) + 1
  FOR each word and count:
    IF count > 1: DISPLAY word and count
END
```

Python Source Code

```
text = "Python is easy, and Python is useful."
clean_text = text.lower()
for mark in ".,!?;:":
    clean_text = clean_text.replace(mark, " ")

words = clean_text.split()
counts = {}

for word in words:
    counts[word] = counts.get(word, 0) + 1

print("Word frequencies:")
for word, count in counts.items():
    print(word, ":", count)

print("Repeated words:")
for word, count in counts.items():
    if count > 1:
        print(word, "appears", count, "times")
```

Test Cases

Test Input	Expected Result	Purpose
“Python is easy, and Python is useful.”	python: 2; is: 2	Case-insensitive counting
“Hello world”	No repeated words	No duplicates
Empty text	No word counts	Empty input

Conclusion

The program demonstrates string processing, loops, dictionaries, and frequency counting to detect repeated words.

4. Tuple-Based Employee Shift Conflict Detection

Problem Statement

• Store employee shift records as tuples and identify shifts assigned to the same employee that overlap in time.

Objectives

• Represent shift records using tuples.

• Compare time intervals for each employee.

• Report overlapping assignments.

Assumptions

• Each shift is stored as (employee, start_hour, end_hour).

• Times use a 24-hour numeric scale and each shift starts before it ends.

• Two shifts overlap when the later start time is earlier than the earlier end time.

Algorithm

1. Store all shifts in a list of tuples.

2. Compare each pair of shifts.

3. Check whether the employee names match.

4. Check whether their time intervals overlap.

5. Display any conflicts.

Flowchart

START → Read tuple shift records → Compare each pair → Same employee and overlapping times? → Report conflict → STOP

Pseudocode

```
BEGIN
  FOR i from 0 to number_of_shifts - 1
    FOR j from i + 1 to number_of_shifts - 1
      unpack shift i and shift j
      IF same employee AND start1 < end2 AND start2 < end1:
        DISPLAY conflict
END
```

Python Source Code

```
shifts = [
    ("Asha", 9, 13),
    ("Bharat", 10, 14),
    ("Asha", 12, 16),
    ("Charan", 14, 18)
]
conflict_found = False

for i in range(len(shifts)):
    name1, start1, end1 = shifts[i]
    for j in range(i + 1, len(shifts)):
        name2, start2, end2 = shifts[j]
        if start1 >= end1 or start2 >= end2:
            print("Invalid shift record")
            continue
        if name1 == name2 and start1 < end2 and start2 < end1:
            print(f"Conflict: {name1} has shifts "
                  f"{start1}-{end1} and {start2}-{end2}")
            conflict_found = True

if not conflict_found:
    print("No shift conflicts found")
```

Test Cases

Test Input	Expected Result	Purpose
Asha 9–13 and Asha 12–16	Conflict reported	Overlapping same-employee shifts
Asha 9–12 and Asha 12–16	No conflict	Back-to-back shifts
Different employees with overlapping times	No conflict	Employee identity checked

Conclusion

The program uses tuples and nested loops to check scheduling conflicts between shifts assigned to the same employee.

5. Food Waste Monitoring System

Problem Statement

• Compare meals prepared, meals served, and leftovers across several days, calculate wastage percentages, and identify days exceeding a waste threshold.

Objectives

• Calculate leftover meals.

• Calculate wastage percentage.

• Flag days with high food waste.

Assumptions

• Prepared meals must be greater than zero.

• Meals served and leftovers cannot be negative.

• A wastage rate above 15% is flagged for review.

Algorithm

1. Read daily meal records.

2. Validate the quantities.

3. Calculate leftover meals as prepared minus served when explicit leftovers are not provided.

4. Calculate wastage percentage.

5. Flag days above the threshold and summarize results.

Flowchart

START → Read daily meal records → Validate quantities → Calculate leftovers and wastage percentage → Compare with threshold → Display daily report → STOP

Pseudocode

```
BEGIN
  FOR each day, prepared, served
    IF prepared <= 0 OR served < 0 OR served > prepared:
      DISPLAY invalid record
    ELSE
      leftovers = prepared - served
      waste_percent = leftovers / prepared * 100
      DISPLAY leftovers and waste_percent
      IF waste_percent > 15: DISPLAY review alert
END
```

Python Source Code

```
daily_meals = [
    ("Monday", 200, 180),
    ("Tuesday", 220, 170),
    ("Wednesday", 180, 175),
    ("Thursday", 210, 185)
]
threshold = 15

for day, prepared, served in daily_meals:
    if prepared <= 0 or served < 0 or served > prepared:
        print(day, "- Invalid meal data")
        continue

    leftovers = prepared - served
    waste = leftovers / prepared * 100
    print(f"{day}: prepared={prepared}, served={served}, "
          f"leftovers={leftovers}, waste={waste:.2f}%")
    if waste > threshold:
        print("  Review alert: wastage exceeds 15%")
```

Test Cases

Test Input	Expected Result	Purpose
200 prepared, 180 served	20 leftovers; 10% waste; no alert	Waste calculation
220 prepared, 170 served	50 leftovers; 22.73%; alert	Threshold alert
150 served from 100 prepared	Invalid meal data	Quantity validation

Conclusion

The system measures food waste using daily data and highlights days where wastage exceeds the selected threshold.

6. Library Borrowing and Fine Reconciliation

Problem Statement

• Process library borrowing records, calculate late-return fines, and report books that remain overdue or have been returned.

Objectives

• Calculate overdue days.

• Calculate fines using a daily rate.

• Summarize returned and overdue records.

Assumptions

• Each record contains a book ID, due day, actual return day, and return status.

• A late fine of ■5 per day applies after the due day.

• If a book has not been returned, the current day is used as the actual day for calculating overdue days.

Algorithm

1. Read the borrowing records and current day.

2. Determine the applicable actual day for each record.

3. Calculate overdue days as the positive difference from the due day.

4. Calculate the fine and status.

5. Display each record and the total fines.

Flowchart

START → Read borrowing records → Determine actual/current day → Calculate overdue days and fine → Classify status → Display report and total → STOP

Pseudocode

```
BEGIN
  total_fines = 0
  FOR each book record
    IF returned: actual = return_day
    ELSE: actual = current_day
    overdue = max(0, actual - due_day)
    fine = overdue * 5
    total_fines = total_fines + fine
    DISPLAY book ID, overdue, fine, status
  DISPLAY total_fines
END
```

Python Source Code

```
current_day = 30
records = [
    ("BK101", 25, 28, True),
    ("BK102", 26, 0, False),
    ("BK103", 29, 30, True)
]
rate_per_day = 5
total_fines = 0

for book_id, due_day, return_day, returned in records:
    actual_day = return_day if returned else current_day
    overdue = max(0, actual_day - due_day)
    fine = overdue * rate_per_day
    total_fines += fine
    status = "RETURNED" if returned else "OVERDUE - NOT RETURNED"
    print(f"{book_id}: overdue={overdue} days, "
          f"fine=Rs.{fine}, status={status}")

print("Total fines: Rs.", total_fines)
```

Test Cases

Test Input	Expected Result	Purpose
Due day 25, returned day 28	3 overdue days; ■15 fine	Late return
Due day 26, not returned by day 30	4 overdue days; ■20 fine	Overdue book
Returned on due day	0 days; ■0 fine	On-time return

Conclusion

The program uses tuple records and date-like day values to calculate fines and identify books that still need to be returned.

7. Manufacturing Quality Control Matrix

Problem Statement

• Analyse production quantities and defective units for multiple products and batches, calculate defect rates, and flag batches above a quality threshold.

Objectives

• Calculate defect percentages.

• Validate production and defect counts.

• Identify batches that exceed the defect threshold.

Assumptions

• Each record contains a product, batch, units produced, and defective units.

• Produced units must be positive and defective units must be between zero and produced units.

• A batch is flagged when the defect rate exceeds 5%.

Algorithm

1. Read batch records.

2. Validate produced and defective quantities.

3. Calculate the defect percentage.

4. Compare the rate with 5%.

5. Display the batch status.

Flowchart

START → Read batch records → Validate quantities → Calculate defect percentage → Above 5%? → Flag batch → Display report → STOP

Pseudocode

```
BEGIN
  FOR each product, batch, produced, defective
    IF produced <= 0 OR defective < 0 OR defective > produced:
      DISPLAY invalid
    ELSE
      rate = defective / produced * 100
      IF rate > 5: status = REVIEW
      ELSE: status = ACCEPTABLE
      DISPLAY product, batch, rate, status
END
```

Python Source Code

```
batches = [
    ("Bottle", "B01", 1000, 25),
    ("Cup", "C02", 800, 56),
    ("Plate", "P03", 500, 10)
]
limit = 5

for product, batch, produced, defective in batches:
    if produced <= 0 or defective < 0 or defective > produced:
        print(batch, "- Invalid production data")
        continue

    rate = defective / produced * 100
    status = "REVIEW" if rate > limit else "ACCEPTABLE"
    print(f"{product} {batch}: defect rate={rate:.2f}%, status={status}")
```

Test Cases

Test Input	Expected Result	Purpose
1,000 produced, 25 defective	2.5%; ACCEPTABLE	Below threshold
800 produced, 56 defective	7%; REVIEW	Above threshold
500 produced, 600 defective	Invalid production data	Validation

Conclusion

The quality-control program analyses batch data and flags high defect rates using a clear threshold.

8. Multi-Branch Sales Matrix Comparison

Problem Statement

• Store sales of several products across multiple branches in a nested list, calculate product totals and branch totals, and identify the strongest sales results.

Objectives

• Calculate branch-wise sales totals.

• Calculate product-wise totals.

• Identify the highest-selling product and branch.

Assumptions

• Each row represents a branch and each column represents a product.

• All sales values are non-negative.

• All branches report sales for the same products.

Algorithm

1. Create branch names, product names, and a sales matrix.

2. Validate the matrix values and dimensions.

3. Calculate each branch's total sales.

4. Calculate each product's total across branches.

5. Display the best-performing branch and product.

Flowchart

START → Read sales matrix → Validate data → Calculate branch totals → Calculate product totals → Find maxima → Display report → STOP

Pseudocode

```
BEGIN
  FOR each branch row
    branch_total = sum(row)
    DISPLAY branch total
  FOR each product column
    product_total = sum(column across branches)
    DISPLAY product total
  DISPLAY branch with maximum total
  DISPLAY product with maximum total
END
```

Python Source Code

```
branches = ["North", "South", "East"]
products = ["Laptop", "Tablet", "Phone"]
sales = [
    [12, 18, 25],
    [15, 14, 30],
    [10, 20, 22]
]

if any(value < 0 for row in sales for value in row) or any(
        len(row) != len(products) for row in sales):
    print("Invalid sales matrix")
else:
    branch_totals = []
    for branch, row in zip(branches, sales):
        total = sum(row)
        branch_totals.append(total)
        print(branch, "total sales:", total)

    product_totals = []
    for j, product in enumerate(products):
        total = sum(row[j] for row in sales)
        product_totals.append(total)
        print(product, "total sales:", total)

    print("Best branch:", branches[branch_totals.index(max(branch_totals))])
    print("Best-selling product:", products[product_totals.index(max(product_totals))])
```

Test Cases

Test Input	Expected Result	Purpose
Sales matrix [[12,18,25],[15,14,30],[10,20,22]]	Branch totals 55, 59, 52; South highest; Phone highest	Totals and ranking
Negative sales value	Invalid sales matrix	Validation
Row with missing product value	Invalid sales matrix	Matrix structure

Conclusion

The program demonstrates nested lists, column-wise and row-wise totals, and simple ranking across branches and products.

9. Weekly Electricity Consumption Analysis

Problem Statement

• Analyse daily electricity readings for several appliances, calculate weekly consumption and cost, and flag appliances whose consumption exceeds a limit.

Objectives

• Calculate weekly appliance consumption.

• Estimate electricity cost.

• Identify appliances exceeding a weekly usage threshold.

Assumptions

• Readings are in kWh and cannot be negative.

• The illustrative electricity rate is ■7 per kWh.

• An appliance is flagged when weekly consumption exceeds 20 kWh.

Algorithm

1. Store appliance names and seven daily readings.

2. Validate the readings.

3. Calculate each appliance's weekly consumption and cost.

4. Compare consumption with the threshold.

5. Display appliance-wise results and the overall total.

Flowchart

START → Read appliance readings → Validate values → Calculate weekly units and cost → Check threshold → Display appliance and total summary → STOP

Pseudocode

```
BEGIN
  total_units = 0
  FOR each appliance and daily_readings
    IF any reading < 0: DISPLAY invalid
    ELSE
      weekly = sum(daily_readings)
      cost = weekly * rate
      total_units = total_units + weekly
      DISPLAY weekly, cost
      IF weekly > 20: DISPLAY high-usage alert
  DISPLAY total_units and total cost
END
```

Python Source Code

```
rate = 7
appliances = {
    "Fan": [2, 2, 2, 2, 2, 2, 2],
    "Computer": [3, 3, 4, 3, 4, 2, 2],
    "Heater": [4, 4, 4, 4, 4, 4, 4]
}
total_units = 0

for appliance, readings in appliances.items():
    if any(value < 0 for value in readings):
        print(appliance, "- Invalid reading")
        continue

    weekly = sum(readings)
    cost = weekly * rate
    total_units += weekly
    print(f"{appliance}: {weekly} kWh, cost=Rs.{cost}")
    if weekly > 20:
        print("  High-usage alert")

print("Total weekly units:", total_units)
print("Total weekly cost: Rs.", total_units * rate)
```

Test Cases

Test Input	Expected Result	Purpose
Fan uses 2 kWh for 7 days	14 kWh; ■98; no alert	Weekly total and cost
Heater uses 4 kWh for 7 days	28 kWh; ■196; alert	Threshold check
Negative reading	Invalid reading	Validation

Conclusion

The program uses lists and dictionary records to calculate weekly electricity usage, estimate costs, and flag high consumption.

10. Campus Transport Capacity Planner

Problem Statement

• Calculate the number of buses required for each route based on passenger demand and bus capacity, and identify any passengers who cannot be accommodated.

Objectives

• Calculate buses needed per route.

• Check the capacity of the available fleet.

• Report unserved passengers when buses are insufficient.

Assumptions

• Passenger demand is a non-negative integer.

• Each bus can carry 40 passengers.

• The sample has a limited number of buses available for each route.

Algorithm

1. Read route passenger demand and available buses.

2. Calculate required buses using ceiling division.

3. Calculate available passenger capacity.

4. Determine unserved passengers if capacity is insufficient.

5. Display the route plan.

Flowchart

START → Read route demand and available buses → Calculate buses needed → Calculate available capacity → Determine shortage → Display route plan → STOP

Pseudocode

```
BEGIN
  FOR each route
    IF passengers < 0 OR buses_available < 0: DISPLAY invalid
    ELSE
      buses_needed = ceiling(passengers / capacity)
      available_capacity = buses_available * capacity
      unserved = max(0, passengers - available_capacity)
      DISPLAY buses_needed, unserved
END
```

Python Source Code

```
routes = [
    ("Route A", 95, 3),
    ("Route B", 130, 2),
    ("Route C", 60, 2)
]
capacity = 40

for route, passengers, available_buses in routes:
    if passengers < 0 or available_buses < 0:
        print(route, "- Invalid data")
        continue

    buses_needed = (passengers + capacity - 1) // capacity
    available_capacity = available_buses * capacity
    unserved = max(0, passengers - available_capacity)

    print(f"{route}: passengers={passengers}, "
          f"buses needed={buses_needed}, available buses={available_buses}, "
          f"unserved passengers={unserved}")
```

Test Cases

Test Input	Expected Result	Purpose
95 passengers, 3 buses available	3 buses needed; 0 unserved	Enough capacity
130 passengers, 2 buses available	4 buses needed; 50 unserved	Capacity shortage
Negative passenger count	Invalid data	Input validation

Conclusion

The planner compares passenger demand with bus capacity and identifies routes that require additional transport.